



Análise e Desenvolvimento de Sistemas (ADS)

Centro de Educação a Distância (CEAD)

PROGRAMAÇÃO EM AMBIENTE DE REDE [ADS0030]

Prof. Francisco Ferreira de Mendonça Júnior

Relatório Técnico do Estudo Guiado 02

Nome completo: Everlir Richardson da Silva

Matrícula: 2023009842

Nome completo: Genildo da Silva Ferreira

Matrícula: 2025013782

Nome completo: Jefferson da Rocha Teodoro

Matrícula: 2025014000

Nome completo: Jefferson Rodrigues de Oliveira

Matrícula: 2025013432

Nome completo: José Valbério da Silva Sousa

Matrícula: 2023009691

Nome completo: Maria Fernanda Sousa Silva

Matrícula: 2025019580

Nome completo: Weber Fernandes da Silva

Matrícula: 2025019356

Nome completo: Yan Brasil Angelim de Brito

Matrícula: 202501902

Sumário

1) Pesquisas Iniciais (Seção 0).....	3
2) Questões para Análise (Seção 9).....	5
3) Síntese Final	7

1) Pesquisas Iniciais (Seção 0)

Pesquisa 4.1: Qual a diferença técnica fundamental entre Edge Computing e Fog Computing? Em qual camada o seu Gateway se encaixa melhor?

- **Resposta:** A diferença fundamental reside na localização física e na abrangência do processamento. **Edge Computing** (Computação de Borda) foca em processar os dados o mais próximo possível da fonte geradora (diretamente nos sensores, atuadores ou em *gateways* adjacentes), reduzindo a latência ao mínimo absoluto. Já a **Fog Computing** (Computação em Névoa), um conceito impulsionado pela Cisco, é uma arquitetura mais ampla que cria uma camada intermediária de rede (LAN, roteadores, servidores locais) entre a borda e a nuvem, distribuindo o armazenamento e o processamento ao longo dessa "névoa".

- **Camada do nosso Gateway:** O nosso script *gateway.py* se encaixa melhor na camada de **Edge Computing** (atuando como um *Edge Gateway*). Ele está posicionado na extremidade da rede local, recebendo os dados brutos via broker MQTT diretamente dos sensores simulados e aplicando a regra de negócio (média e caminhos críticos) imediatamente no local antes de qualquer trânsito externo para a nuvem.

Pesquisa 4.2: No protocolo MQTT, o que são os níveis de QoS (Quality of Service) 0, 1 e 2? Qual nível seria mais adequado para o envio de temperatura constante e qual seria mais adequado para o alerta de emergência?

- **Resposta:** Os níveis de QoS do MQTT definem a garantia de entrega das mensagens entre o remetente e o receptor:

- **QoS 0 (At most once - No máximo uma vez):** É o método "dispare e esqueça". A mensagem é enviada sem exigir confirmação. Pode haver perda de pacotes, mas é o nível mais rápido e com menor consumo de rede.

- **QoS 1 (At least once - Pelo menos uma vez):** Garante que a mensagem chegue ao destino, pois exige uma confirmação (*PUBACK*). Contudo, em redes instáveis, a mesma mensagem pode ser entregue em duplicidade.

- **QoS 2 (Exactly once - Exatamente uma vez):** É o nível mais seguro e mais lento. Utiliza um *handshake* de quatro etapas para garantir que a mensagem chegue perfeitamente e sem duplicações.

- **Aplicação no projeto:** Para o **envio de temperatura constante** a cada 100ms, o **QoS 0** é o mais adequado, pois a alta frequência torna a perda de um único pacote irrelevante (logo em seguida chega outro) e economiza processamento. Já para o **alerta de emergência** (Caminho Crítico), o ideal seria utilizar **QoS 1 ou QoS 2**, garantindo que o alarme de superaquecimento jamais se perca na rede em caso de falhas de conexão.

Pesquisa 4.3: O que é uma "Janela de Tempo" (Time Window) na agregação de dados? Explique a diferença entre uma Tumbling Window (Janela Saltitante) e uma Sliding Window (Janela Deslizante).

• **Resposta:** Uma "Janela de Tempo" é uma técnica de processamento de fluxo de dados (*Stream Processing*) que divide um fluxo contínuo e infinito de eventos em recortes temporais finitos para aplicar cálculos (como médias, somas ou contagens).

- **Tumbling Window (Janela Saltitante):** Divide os dados em segmentos de tempo fixos, adjacentes e **não sobrepostos**. Por exemplo, janelas de 1 segundo (0-1s, 1-2s, 2-3s). Cada evento gerado pertence a uma e apenas uma janela. Quando a janela acaba, ela calcula o resultado, limpa o buffer e a próxima janela começa do zero. **Esta foi a janela implementada no nosso Gateway.**

- **Sliding Window (Janela Deslizante):** Opera com intervalos sobrepostos. Ela é definida por um tamanho de janela e um intervalo de "deslizamento" (avanço). Por exemplo, uma janela de 10 segundos que atualiza a cada 2 segundos. Isso significa que um mesmo dado pode ser processado e pertencer a múltiplas janelas diferentes à medida que o tempo avança, sendo útil para observar tendências progressivas mais suaves.

2) Questões para Análise (Seção 9)

I. Economia de Banda

Calcule matematicamente a porcentagem de economia de banda alcançada pelo seu Gateway.

• **Resposta:** A arquitetura proposta utiliza o processamento na borda (Edge Analytics) por meio de uma Tumbling Window (janela temporal) de 1 segundo. No cenário simulado, temos 30 sensores operando com um intervalo de envio de 100ms (0,1s). Sendo assim, cada sensor envia 10 mensagens por segundo, totalizando 300 mensagens brutas recebidas pelo Gateway localmente a cada segundo. Ao invés de trafegar todas essas mensagens para a nuvem, o Gateway consolida os dados em uma média aritmética e envia apenas 1 pacote contendo o resumo estatístico. A economia de banda gerada é calculada da seguinte forma:

$$Economia = \left(1 - \frac{1}{300}\right) \times 100 \approx 99.67\%$$

II. Latência de Alerta

No seu teste, quanto tempo levou entre o sensor detectar a anomalia e o e-mail chegar (ou ser processado pelo Gateway)? Por que esse tempo é menor do que se o sensor tivesse que enviar o dado para a Nuvem e a Nuvem disparar o e-mail?

• **Resposta:** O processamento da anomalia (temperatura > 60°C) ocorre na ordem de milissegundos. Isso acontece porque a verificação do "Caminho Crítico" foi implementada diretamente na função de *callback* de recebimento do protocolo MQTT (*on_message*), reagindo instantaneamente à chegada do pacote sem aguardar o fechamento da janela de 1 segundo. Esse tempo é drasticamente menor do que uma arquitetura centralizada na nuvem porque elimina o *Round-Trip Time* (RTT). Enviar para a nuvem exigiria que o dado passasse pela rede externa, sofresse com a latência da internet, fosse processado por um servidor remoto e, só então, gerasse o gatilho de alerta, adicionando riscos de atraso que são inaceitáveis para a segurança de máquinas industriais.

III. Falha de Conectividade

Se a conexão com a internet cair, seu Gateway para de funcionar? Quais funcionalidades continuariam ativas localmente?

• **Resposta:** Não, o Gateway continua operando de forma autônoma. Devido à sua natureza de *Edge Computing*, a dependência da internet restringe-se exclusivamente ao envio do resumo para o banco de dados (Firebase). Se houver interrupção de rede, as seguintes funcionalidades locais continuam 100% ativas: a recepção dos dados dos sensores via broker MQTT local, o armazenamento no buffer em memória, o cálculo das médias pela *thread* cronometrada e, principalmente, a detecção de anomalias no Caminho Crítico, garantindo que a fábrica continue sendo monitorada contra superaquecimentos mesmo totalmente *offline*.

IV. Escalabilidade

Se em vez de 30 sensores, tivéssemos 3.000 sensores enviando dados a cada 10ms, qual seria o principal gargalo do seu código atual? Como você resolveria isso usando conceitos de sistemas distribuídos?

• **Resposta:** Com 3.000 sensores a 10ms, o sistema receberia 300.000 mensagens por segundo. O principal gargalo da implementação atual seria a saturação de concorrência. O uso de um único `threading.Lock()` geraria uma fila de espera massiva (*lock contention*) bloqueando a recepção de novas mensagens. Além disso, a memória do `buffer_dados` estouraria rapidamente e um único broker Mosquitto não suportaria a carga de conexões simultâneas.

Para resolver isso utilizando conceitos de sistemas distribuídos, a solução envolveria:

- **Particionamento de Dados:** Substituir o MQTT simples por uma plataforma de *streaming* distribuída como o Apache Kafka.

- **Balanceamento de Carga e Múltiplos Gateways:** Dividir a fábrica em zonas físicas (ex: 500 sensores por zona), atribuindo um Gateway independente para cada setor (arquitetura *Fog Computing* descentralizada).

- **Caches Distribuídos:** Utilizar ferramentas como Redis para gerenciar o estado do buffer, em vez de depender da memória local de um único script Python.

3) Síntese Final

A realização da Prática 02 permitiu consolidar os conceitos fundamentais de Sistemas Industriais IoT e Computação de Borda (*Edge Computing*). Através do desenvolvimento de um simulador multithread e de um Smart Gateway em Python, foi possível observar, na prática, os imensos benefícios operacionais e de rede ao descentralizar o processamento de dados.

A arquitetura desenvolvida demonstrou altíssima eficiência e resiliência. A implementação estatística baseada em uma *Tumbling Window* de 1 segundo no Gateway local resultou em uma impressionante economia de banda de aproximadamente **99,67%**. Essa abordagem não apenas mitiga o risco de congestionamento na rede, mas também reduz drasticamente os custos de tráfego e armazenamento em nuvem, enviando ao banco de dados (Firebase Realtime Database) apenas resumos consolidados para histórico a longo prazo.

O destaque do sistema em termos de segurança industrial foi a validação do **Caminho Crítico**. Ao processar dados de temperatura na borda, o Gateway garantiu a identificação instantânea de anomalias (acima de 60°C) na própria função de *callback* do protocolo MQTT, eliminando a latência de comunicação com a internet e permitindo o disparo de alertas em milissegundos. Adicionalmente, a arquitetura provou-se resiliente a falhas de conectividade externa, garantindo que o monitoramento local de emergências continuasse operando mesmo em cenários totalmente *offline*.

Por fim, o rigor técnico aplicado ao desenvolvimento da solução — evidenciado pelo uso de `threading.Lock` para evitar condições de corrida em operações assíncronas e pela adoção das melhores práticas de segurança cibernética utilizando variáveis de ambiente (`.env`) para compartimentar credenciais — assegurou a entrega de um software robusto. O projeto cumpre de forma exitosa e integral todos os requisitos propostos, servindo como um laboratório prático indispensável para a compreensão da engenharia de redes voltada para a Indústria 4.0.